MobilePA-Bench: Benchmarking Mobile Planner Agents on Complex Real-World Tasks

Yi Zhu*, Xiongwei Wu*, Qiyi Wang‡, Tingyu Qu, Jiajun Liu, Sihan Cao,
Long Chen, Weigao Sun, Feida Zhu§, Yiran Zhong§, Steven HOI

MAI Team, Alibaba Token Hub, Alibaba Group

https://github.com/Tongyi-MAI/MobilePA-Bench

Abstract

Mobile planning requires a central agent to coordinate tools, persistent user context, reusable procedures, and specialized sub-agents while adapting to execution feedback. Evaluating these workflows requires distinguishing task completion from compliance with prescribed resource and procedural requirements. We introduce **MobilePA-Bench**, a benchmark comprising 1,705 tasks across eight query task domains and 212 mobile tools in a stateful simulated environment. It combines three features: (1) **unified capability coverage**, profiling *Tool Use*, *Memory Usage*, *Skill Usage*, and *Sub-agent Collaboration* under a common mobile protocol; (2) **evidence-aligned verification**, matching each task to a fixed tool-call, state-change, or interaction-behavior checker rather than imposing one completion criterion on all tasks; and (3) **capability-gated evaluation**, requiring task completion alongside prescribed memory checks and, in skill-only routing, complete route loading before business-tool use. With MobilePA-Bench, we systematically evaluate open- and closed-weight LLMs, revealing capability-specific weaknesses in complex mobile task execution. Fine-grained analyses distinguish failures in resource retrieval and procedural compliance from downstream execution errors, providing diagnostic insights beyond aggregate task-success rates. We further train a Qwen3.8-27B baseline with online reinforcement learning in the sandbox, whose improved performance provides initial evidence of the environment's utility for planner development.

1. Introduction

Mobile agents are evolving from single-step interface operators into personal assistants that handle multi-step tasks across applications. Arranging travel around a user's preferences, for example, may require querying application state, invoking tools, retrieving personal information, following reusable procedures, and delegating operations to specialized sub-agents (Figure 1). A central *mobile planner* must coordinate these resources, order its actions, and revise its plan from execution feedback. Evaluating mobile planning therefore requires not only measuring task completion, but also identifying weaknesses in the capabilities involved.

Existing benchmarks offer complementary views of agent capabilities. GUI benchmarks assess task completion through application interfaces [1, 2]; DroidCall evaluates mobile API selection and argument generation [3]; and AppWorld, τ^3-bench, and Toolathlon assess interactive, stateful tool execution [4–6]. MemoryArena, SkillsBench, and MultiAgentBench examine memory, skills, and

* Equal contribution. ‡ Research intern. § Project lead.

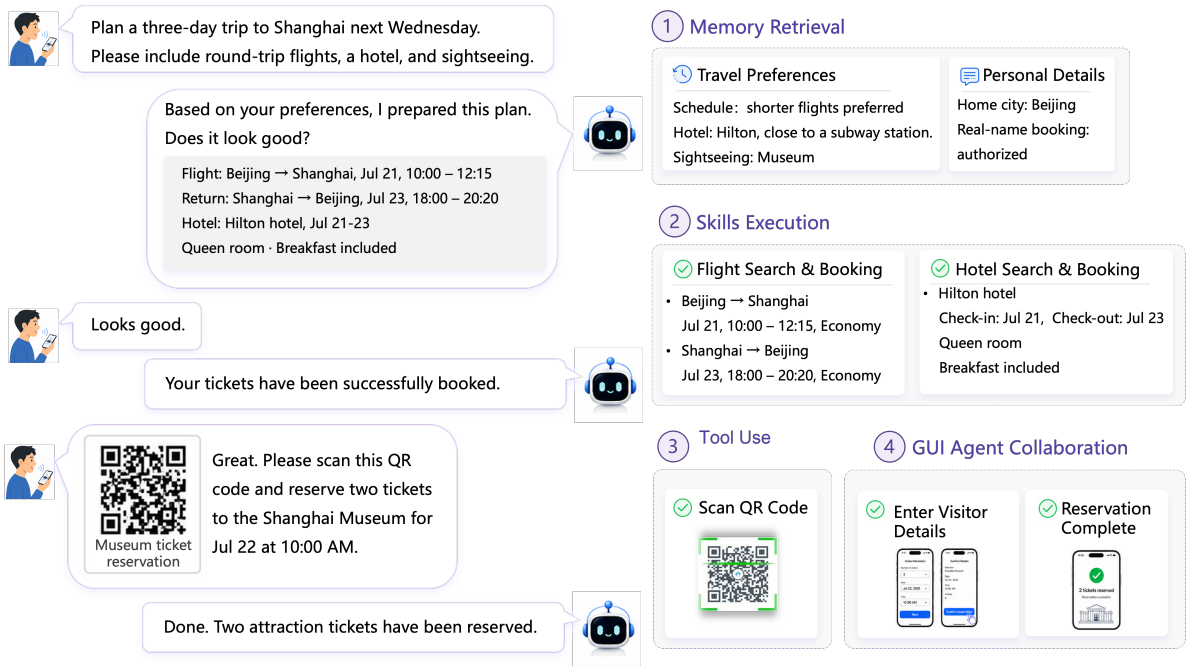


Figure 1: **An illustrative mobile workflow involving four planner capabilities.** The central planner invokes mobile tools (**Tool Use**), retrieves user preferences (**Memory Usage**), loads reusable procedures (**Skill Usage**), and delegates interface operations to a GUI agent, one of its available sub-agents (**Sub-agent Collaboration**).

multi-agent collaboration, respectively [7–9]. Their environments, action interfaces, and success criteria differ. For central mobile planners, we seek a capability profile under a common environment and protocol that connects task outcomes with capability-use evidence. Outcome checks alone cannot establish whether prescribed resource or procedural requirements were met; invocation checks alone cannot establish subsequent task completion.

To this end, we introduce **MobilePA-Bench**, a stateful, executable benchmark for central mobile planners, comprising 1,705 tasks across eight query task domains and 212 mobile tools. Its **unified capability coverage** spans four dimensions: *Tool Use* tests tool invocation, argument grounding, and cross-step dependencies; *Memory Usage* examines retrieval, maintenance, and memory-related task execution; *Skill Usage* tests completion under skill-only and mixed tool-skill access; and *Sub-agent Collaboration* evaluates routing, contextual handoff, and follow-up, with GUI agents among the available specialists. These task subsets run through shared interfaces in a simulated environment with persistent application state, operation dependencies, and structured feedback, enabling analysis of overall performance and individual capabilities under a common mobile protocol.

To ground evaluation in inspectable execution evidence, MobilePA-Bench combines **evidence-aligned verification** with **capability-gated evaluation**. Each task receives a fixed primary checker during annotation, using tool calls, state changes, or interaction behavior according to its completion semantics. Capability gates check prescribed memory access and action constraints, or timely skill loading in skill-only routing; mixed tool-skill routing also accepts valid direct-tool solutions. Reporting these checks alongside task outcomes separates unmet requirements from downstream execution failures without penalizing valid alternative paths in the mixed setting. The benchmark thus identifies observable failure stages beyond aggregate task-success rates, supporting diagnosis of capability-specific weaknesses in complex workflows.

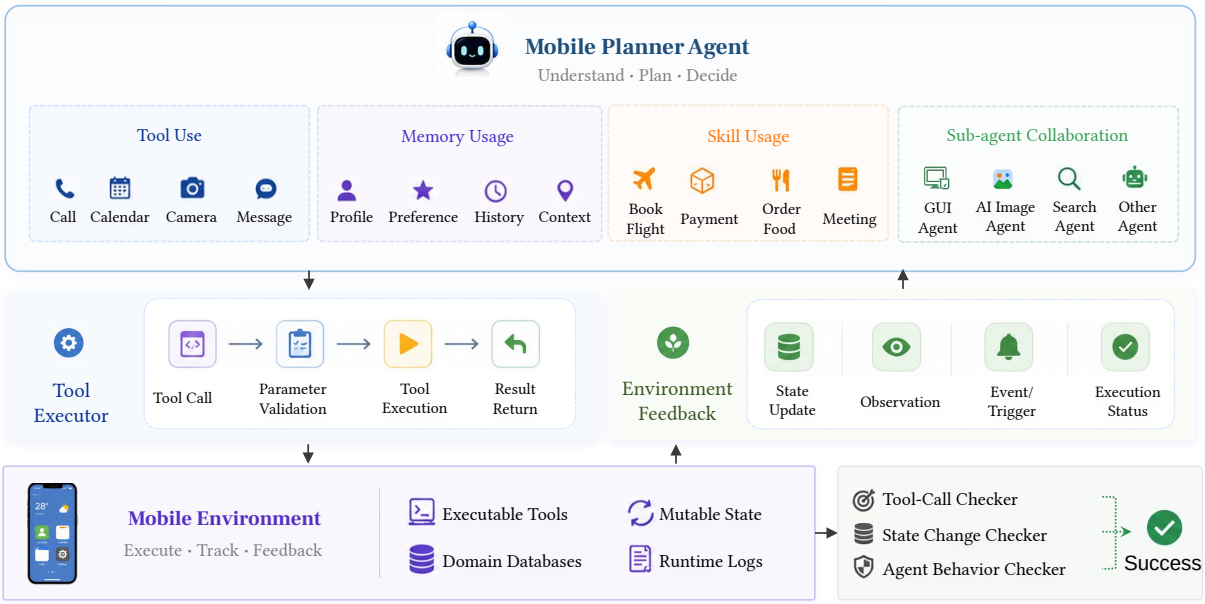


Figure 2: **Overview of MobilePA-Bench.** A central mobile planner acts through unified interfaces for tools, memory, skills, and specialized sub-agents. The stateful mobile environment executes each action and returns structured feedback, while the assigned verifier evaluates the resulting calls, state changes, or interaction behavior.

With MobilePA-Bench, we systematically evaluate open- and closed-weight LLMs on complex mobile tasks. Fine-grained analyses reveal difficulties in argument grounding, dependency planning, and boundary handling. Retrieving required memories or loading skills does not ensure subsequent task completion, while selecting an appropriate sub-agent does not ensure that the full interaction requirements are met. We further train a `Qwen3.8-27B` baseline with online reinforcement learning in the sandbox and observe improved benchmark performance, providing initial evidence that this evaluation environment can also support planner development.

2. Related Work

Mobile and Tool-Using Agent Benchmarks Agent benchmarks cover GUI-based task completion [1, 2], mobile API-call prediction [3], and interactive tool execution [4–6]. LlamaTouch evaluates on-device execution through essential application states [10]; VitaBench studies multi-turn tool use and changing user intent in service simulations [11]. MobilePA-Bench instead isolates central mobile planning through structured tools and delegation in a stateful sandbox.

Memory, Skills, and Agent Delegation LongMemEval and BFCL v4 assess memory [12, 13], while MemoryArena evaluates memory acquisition and reuse across interdependent agentic sessions [7]. SkillsBench measures skill augmentation, and MultiAgentBench evaluates collaboration [8, 9]. MobilePA-Bench combines mobile-task completion with annotated resource-access and procedural requirements. Its gates measure observable compliance, not the causal contribution of resource contents; its Sub-agent score concerns routing and handoff rather than downstream execution.

Task Verification and Evaluation AppWorld and τ-bench verify database outcomes [4, 14], while BFCL uses call- and state-based checks [13]. MCPAgentBench evaluates tool selection with distractors and simulated MCP tools [15]. MobilePA-Bench combines task-specific call, state, or behavior checkers with capability gates.

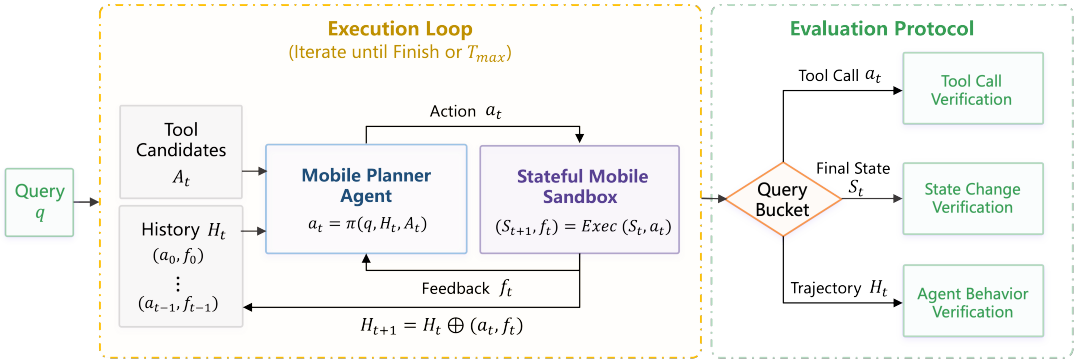


Figure 3: **Closed-loop execution and verification.** The planner acts, the sandbox updates state and returns feedback, and the assigned checker evaluates the terminal trajectory and state.

3. MobilePA-Bench

MobilePA-Bench separates three evaluation questions: which planner capabilities are covered, what evidence establishes task completion, and which capability-specific conditions must also hold. A shared executable environment supports unified capability coverage; evidence-aligned checkers assess completion; and capability gates enforce applicable memory and skill-only routing requirements. Figure 2 summarizes the execution framework.

3.1. Executable Mobile Environment

The stateful mobile simulator (Figure 2) evaluates central-planner routing and cross-step coordination, with a GUI agent available as one specialized sub-agent. It preserves application records, operation dependencies, and structured failures; visual perception, UI dynamics, and device-level execution are outside its scope.

Task Formulation. A mobile task is $\tau = (q, \mathcal{S}_0, \mathcal{H}_0, \mathcal{A}_0)$, where q is the user request, $\mathcal{S}_0$ the initial state, $\mathcal{H}_0$ the optional dialogue history, and $\mathcal{A}_0$ the active candidate action set. At step t, the planner acts and the sandbox updates the state and public interaction history:

$$\begin{aligned} a_t &= \pi(q, \mathcal{H}_t, \mathcal{A}_t), \\ (\mathcal{S}_{t+1}, f_t) &= \mathrm{Exec}(\mathcal{S}_t, a_t), \\ \mathcal{H}_{t+1} &= \mathcal{H}_t \oplus (a_t, f_t). \end{aligned} \tag{1}$$

Execution ends when the planner emits `Finish` or reaches $T_{\max}$; the assigned verifier then evaluates $\mathcal{H}_T$ and $\mathcal{S}_T$ (Figure 3).

Stateful Mobile Environment. The sandbox realizes Exec through three layers: a **Tool Schema** presented to the planner, **Tool Implementation Code** that enforces parameter and execution rules, and a **Shared Stateful Database** that maintains application records and operational logs. We represent its state and returned feedback as

$$\begin{aligned} \mathcal{S}_t &= \langle \mathcal{D}_t, O_t \rangle, \\ f_t &= \langle \mathrm{Status}, \mathrm{ErrorType}, \mathrm{Payload} \rangle. \end{aligned} \tag{2}$$

Each call validates its arguments, updates the relevant application state, records the operation, and returns a structured result or error. Resettable database snapshots and retained logs enable deterministic replay, while controlled friction such as missing arguments, permission blocks, and entity ambiguity

tests whether planners can recover from execution feedback. All mobile, memory, skill-loader, and sub-agent resources follow the same function-calling protocol. Appendix C details the action interfaces and an implementation example.

Reference Harness. The reference harness assembles the request, history, and candidate schemas, dispatches model-generated calls, and appends feedback until termination. The model selects actions; the environment maintains state and evaluation evidence.

3.2. Unified Capability Coverage

We construct four task subsets to assess complementary requirements of a central mobile planner within one executable benchmark. Each instance specifies an initial state, candidate actions, and a capability target. The shared environment and function-calling interface let the same planner be evaluated across Tool Use, Memory Usage, Skill Usage, and Sub-agent Collaboration, yielding a capability profile rather than an isolated score. The subsets retain capability-specific requirements.

Tool Use. Tool Use grounds user intent in valid tools and arguments while handling dependencies and stateful feedback. Starting from human-curated mobile requests, we introduce obfuscated references, permission constraints, intermediate state changes, and runtime errors. The tasks cover parameter grounding, dependency planning, reference and state tracking, intent revision, and boundary handling.

Memory Usage. Memory Usage enables personalized execution when requests leave user-specific information implicit. We construct coherent profile worlds of identities, habits, routines, and preferences, then derive tasks that retrieve and apply those records or update persistent memory. Retrieval tasks annotate the required records and downstream objective.

Skill Usage. Skill Usage evaluates reusable procedures under two action spaces. Each task has one or more acceptable skill routes, comprising one or several skills. Loading a skill exposes its instructions and bound tools. Skill-Only Routing (SOR) requires a complete route to be loaded before business-tool use, followed by task completion. Mixed Tool-Skill Routing (MTSR) also exposes direct tools and accepts valid direct-tool, skill-based, or mixed solutions; skill loading is not mandatory.

Sub-agent Collaboration. Sub-agent Collaboration is required when specialized components handle GUI interaction, visual processing, conditional monitoring, or interactive practice. We construct requests around these routes and annotate the valid sub-agent and contextual handoff payload, isolating planner-side routing and handoff quality from downstream specialist execution.

3.3. Evidence-Aligned Verification

Evidence-aligned verification matches the completion criterion to the task, rather than applying one trajectory- or state-matching rule to every query. Exact trajectory matching can reject valid alternative plans, whereas terminal-state checks can miss required operations, handoffs, or user-facing interactions. Each query therefore receives a fixed primary checker during annotation, as shown in Figure 4. This assignment determines which evidence establishes completion; the additional capability conditions are defined in Section 3.4.

Let $b(q) \in \{\text{tool}, \text{state}, \text{behavior}\}$ denote the bucket assigned to query q, and let $C_{b(q)}(\mathcal{H}_T, \mathcal{S}_T) \in \{0, 1\}$ denote its completion verdict, including applicable execution-validity checks. **Tool Call** checks required tools, arguments, and ordering for deterministic operations; **State Change** checks the annotated terminal database delta when multiple paths are valid; and **Agent Behavior** applies task-specific rubrics to observable routing, handoff, clarification, or recommendation behavior. The assignment is

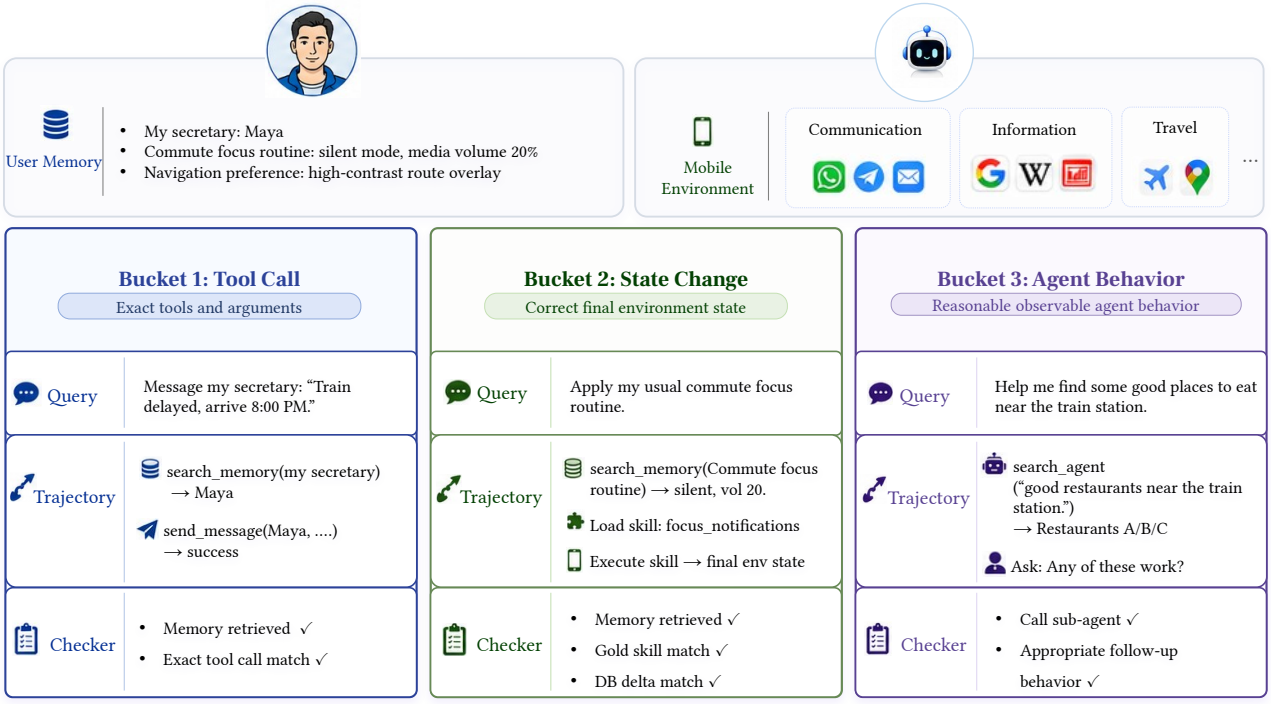


Figure 4: **Evidence-aligned verification in MobilePA-Bench.** Each query is assigned a fixed primary checker for tool calls, terminal state changes, or observable agent behavior.

fixed before model evaluation and is not selected from the model's output. This preserves explicit operational requirements where needed while allowing terminal-state evidence for tasks with multiple valid paths. The checker outputs distinguish call-level, state-level, and interaction-level failures.

3.4. Capability-Gated Evaluation

Capability gates enforce requirements specific to each evaluation setting. The Memory gate combines required checks, including annotated-record retrieval when required, with task-declared action and argument constraints. In SOR, the Skill gate requires a complete acceptable route to be loaded before the first non-skill tool call. MTSR instead accepts any valid completion using the available tools and skills, without a loading gate.

For a fixed setting, let $R_q \subseteq \{\text{mem}, \text{skill}\}$ denote the applicable gates: Skill tasks include skill in SOR but use $R_q = \varnothing$ in MTSR. The task outcome is

$$\text{Succ}(q) = C_{b(q)}(\mathcal{H}_T, \mathcal{S}_T) \wedge \bigwedge_{r \in R_q} g_r(q), \tag{3}$$

with an empty conjunction equal to one. Skill completion includes execution validity and both behavior judges' verdicts; MTSR rubrics accept valid alternative paths rather than requiring the annotated skill route. Memory addition, replacement, and removal tasks do not require retrieval unless specified; their outcome and any applicable action constraints still apply. Gates verify observable compliance, not causal use of retrieved content. Route loading remains a diagnostic in MTSR, not a success requirement.

3.5. Benchmark Statistics and Scoring

MobilePA-Bench contains 1,705 distinct mobile queries and yields 1,905 scored trajectories per model: 1,040 Tool Use queries; 376 Memory queries covering retrieval-dependent execution or memory updates; 200 Skill queries evaluated under both SOR and MTSR, yielding **400 skill-evaluation instances**; and 89 Sub-agent queries covering routing, contextual handoff, and follow-up. SOR requires skill loading and completion, whereas MTSR accepts valid alternative paths. Scores assess executed task outcomes under each setting's requirements, not merely plausible final responses. The environment provides 212 tools across 13 tool domains, distinct from the query-level taxonomy.

Score Aggregation. Overall is the weighted sum of Tool Use, Memory, Skills, and Sub-agent scores with fixed weights of 50%, 20%, 20%, and 10%, respectively. The authors set these weights with reference to query counts and qualitative judgments of task complexity and difficulty, rather than a calibrated difficulty model or measured user utility. Overall is a benchmark index; we therefore report the full capability vector and weight-sensitivity diagnostics alongside it. Tool Use aggregates its task categories; Memory uses gated end-to-end success; Skills averages the SOR and MTSR success rates, with a loading gate only in SOR; and Sub-agent measures routing-and-handoff success. All scores retain the full denominator, counting missing or invalid predictions as failures.

4. Experiments

We first examine the capability profiles exposed by unified coverage, then use the recorded checks to separate capability-gate failures from downstream completion failures. Reliability analyses assess score variation and weight sensitivity. Finally, small-scale online RL examines the sandbox's utility for planner development.

4.1. Evaluation Protocol

We evaluate 14 frontier LLMs on all 1,705 MobilePA-Bench tasks. Because each of the 200 Skill Usage tasks is executed under both Skill-Only Routing (SOR) and Mixed Tool-Skill Routing (MTSR), the evaluation contains 1,905 scored trajectories per model. Candidate lists are supplied with the tasks, rather than retrieved online by the evaluated model. Non-Skill tasks expose 15 tools. Skill evaluation uses 20 candidate skills in SOR and 15 direct tools plus five candidate skills in MTSR; loading a skill exposes its bound tools. Thus, this comparison changes both direct-tool access and the skill candidate set, rather than isolating the effect of direct tools. Every model uses the reference harness and the same function-calling interface until emitting `Finish` or reaching $T_{\max} = 15$. Missing or invalid predictions count as failures. Each task uses its pre-assigned primary checker and any required capability gate; API-judged outcomes use a fixed unanimous-AND decision from two judges. Appendix B.1 reports reference-tool accessibility and provenance checks; these do not establish open-ended retrieval quality.

4.2. Overall Performance

Table 1 reports the recorded capability and Overall scores under the setting-specific success criteria and fixed aggregation defined in Sections 3.4–3.5.

`GPT-6-Astra` has the highest Overall point estimate at 76.87%, while nine of the 14 models score below 70%. Its observed lead over `Claude-Opus-5` is 1.16 percentage points. The dimension leaders differ: `Claude-Opus-5` leads Tool Use, `Claude-Fable-5` leads Memory Usage and Sub-agent Collaboration, and `GPT-6-Astra` leads Skill Usage.

Across models, the mean scores are 73.31% for Tool Use, 66.13% for Memory Usage, 70.14% for Skill

Model	Size / Access	Overall	Tool Use	Memory	Skills	Sub-agent
GPT-6-Astra	Closed source	**76.87**	75.77	74.73	**93.25**	53.93
Claude-Opus-5	Closed source	75.71	**77.60**	71.81	83.00	59.55
Claude-Fable-5	Closed source	74.53	77.21	**76.33**	69.00	**68.54**
Gemini-3.6-Flash	Closed source	69.61	74.71	69.68	65.75	51.69
Gemini-3.1-Pro	Closed source	68.16	75.38	65.69	55.75	61.80
Claude-Opus-4.8	Closed source	66.05	72.40	56.38	69.25	47.19
Qwen-3.7-Max	Closed source	64.89	72.31	64.10	53.75	51.69
Seed-2.1-Pro	Closed source	64.89	70.29	57.18	64.00	55.06
GLM-5.3	744B-A40B	73.88	76.44	72.07	77.00	58.43
Qwen-3.8-Max	2.4T-A95B	71.77	73.65	73.14	75.75	51.69
Kimi-K3	2.8T-A104B	69.64	71.54	71.01	74.75	47.19
GLM-5.3-Flash	320B-A18B	68.31	71.35	65.16	68.25	59.55
GLM-5.2	744B-A40B	66.06	73.75	60.37	58.00	55.06

Table 1: **MobilePA-Bench results (%).** Overall uses the fixed 50/20/20/10 weighting in Section 3.5. Bold marks the highest observed score in each column. Sizes use B (billion), T (trillion), and A (active parameters). Full-cohort analyses include GPT-5.6-Sol (Appendix B.2).

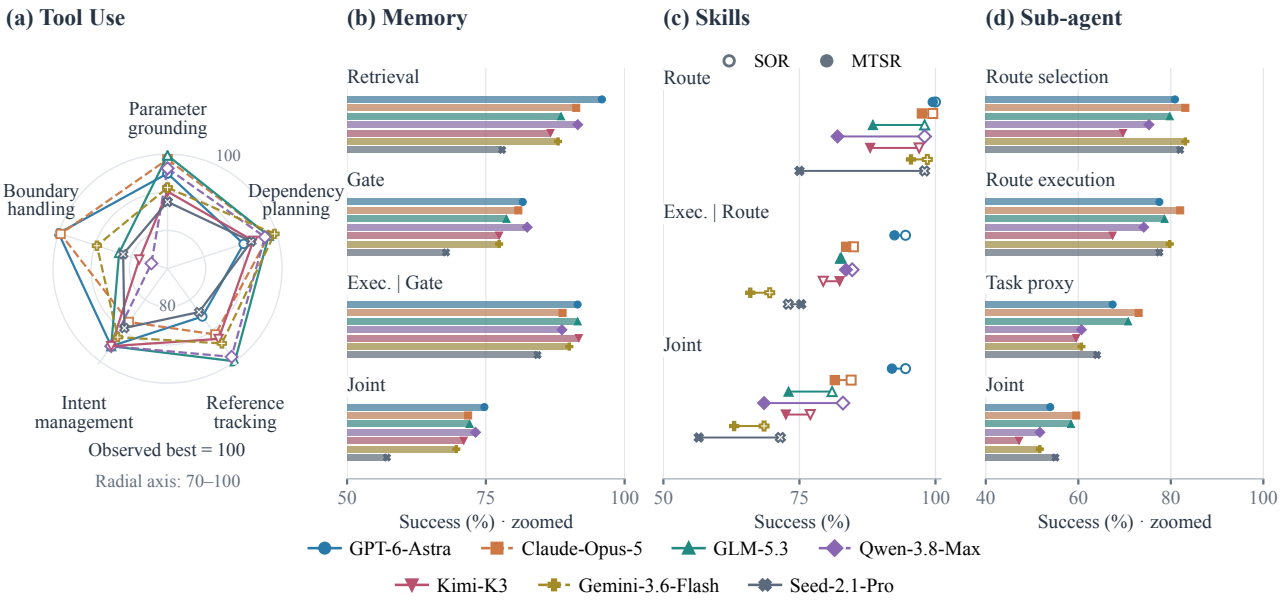


Figure 5: **Fine-grained capability profiles of seven family representatives** (highest Overall per family; Table 1). (a) Tool Use normalized to each subtype's best observed score across 14 models (= 100). (b–d) Rates (%); (c) Skill route diagnostics (SOR: open markers; MTSR: filled).

Usage, and 54.18% for Sub-agent Collaboration. Skill Usage has the widest range (39.50 points), whereas no model leads all four dimensions. Unified coverage therefore exposes planner profiles that an Overall score alone would hide. These are differences on the benchmark's task subsets, not a difficulty-calibrated ranking of the underlying capabilities.

4.3. Fine-Grained Capability Diagnosis

Beyond aggregate scores, MobilePA-Bench examines four complementary capabilities through behavioral categories and execution-level checks. Figure 5 presents seven model-family representatives, revealing both uneven capability profiles and differences between resource access, prescribed behavior,

and task completion. This decomposition identifies weaknesses that aggregate rankings can obscure. Full-cohort summaries and descriptive Skill comparisons are provided in Appendix B.2.

Tool Use: Uneven behavioral profiles. Strong performance in one type of tool use does not imply consistent performance across others. Figure 5(a) visualizes these differences relative to the highest observed score in each subtype. In raw accuracy, `GLM-5.3` and `Qwen-3.8-Max` achieve 94.5% and 93.2% on Reference Tracking, but only 67.0% and 59.8% on Boundary Handling. Parameter Grounding and Dependency Planning also remain below 80% for every selected representative. These results distinguish proficiency in tracking referenced entities from reliability in grounding arguments, satisfying prerequisites, and handling execution boundaries, providing more specific improvement targets than a single Tool Use score.

Memory: Distinguishing unmet requirements from completion failures. Retrieving the required records is only one component of Memory performance. Figure 5(b) separates retrieval, compliance with the complete memory gate, and task completion. Across all seven representatives, more tasks fail the complete gate than pass it but fail completion. For `Qwen-3.8-Max`, these two groups contain 66 and 35 of the same 376 tasks, respectively. This decomposition distinguishes unmet retrieval or action requirements from completion failures among gate-passing trajectories, directing attention to different aspects of memory-supported execution. Retrieval and conditional execution use different task subsets, so the four displayed rates should not be interpreted as a sequential attrition funnel.

Skills: Separating skill routing from task execution. Correct skill loading does not guarantee successful execution. In SOR, the seven representatives achieve Route rates of 97.0%–100.0%, while Joint success ranges from 68.5% to 94.5% (Figure 5(c)). `Gemini-3.6-Flash`, for example, achieves 98.5% Route but only 68.5% Joint success, exposing substantial completion failures despite satisfying the routing requirement. The mixed configuration reveals further differences in routing behavior: `Qwen-3.8-Max`'s ordered-route pass rate changes from 98.0% in SOR to 82.0% in MTSR. However, a lower reference-route pass rate does not establish lower task completion. MTSR accepts valid direct-tool, skill-based, and mixed solutions; Route and Joint diagnose reference-route use, not MTSR task success. Because both tool access and skill candidates differ between settings, this comparison characterizes task-dependent routing rather than isolating direct-tool interference.

Sub-agent: Correct routing is insufficient for complete interaction. Figure 5(d) distinguishes selecting a specialist, executing the routing action, and satisfying the complete task-level interaction requirements. `Claude-Opus-5` and `Gemini-3.6-Flash` both achieve 83.1% route selection, yet their Joint scores are 59.6% and 51.7%, respectively. Thus, similar routing accuracy can conceal different levels of complete interaction success. These checks motivate closer examination of handoff and follow-up behavior rather than treating a correct invocation as successful collaboration. The results concern planner-side interactions over 89 tasks, not downstream specialist execution; the saved task-success proxy is also not an independent measure of handoff quality.

Together, these analyses combine coverage across four capabilities with finer distinctions within each capability. They move evaluation beyond identifying which model scores higher toward identifying which observable behaviors warrant further investigation. The resulting profiles can help prioritize targeted training and validation for argument grounding, dependency handling, memory requirements, skill-supported execution, and delegation behavior, without assuming that the observed differences establish their internal causal mechanisms.

4.4. Reliability and Score Sensitivity

Evaluation Stability. Three complete evaluations of an additional `Qwen3.8-27B` configuration yield Overall scores of 66.60%–67.35% (sample standard deviation 0.38 percentage points), with Memory the

most variable dimension (standard deviation 2.15 points). This model is distinct from `Qwen-3.8-Max` in Table 1; these runs do not establish the rerun variability of all 14 models.

Ranking Sensitivity. We compare the default weighting with equal-dimension and equal-evaluation-instance weighting. Both preserve the top-six order. Equal-dimension weighting shifts five models by at most two positions; instance weighting only swaps ranks 10 and 11. The leading order is stable under these alternatives, but Overall should be read alongside capability-specific scores.

4.5. Online RL Baseline Model

We initialize the baseline from `Qwen3.8-27B` and construct 4,000 training queries disjoint from the benchmark's test queries. We train in the simulated mobile sandbox using Group Relative Policy Optimization (GRPO), with executable tools and structured environment feedback. Rewards are supplied by a learned judge model trained on trajectories collected during preliminary evaluations of API-based models. Table 2 compares the trained checkpoint with the original model's mean over three complete evaluations under the same benchmark protocol.

Overall improves from 66.94% to 69.25%, a gain of 2.31 percentage points. Tool Use, Memory Usage, Skill Usage, and Sub-agent Collaboration increase by 3.06, 0.48, 2.33, and 2.25 points, respectively. Tool Use improves most, while the smaller Memory gain indicates uneven progress across capabilities. These results illustrate the benchmark's dual role in supporting online planner optimization and locating capability-specific improvements. This is a single-checkpoint comparison, not evidence from repeated independent training runs.

Model / Setting	Overall	Tool Use	Memory	Skills	Sub-agent
Qwen3.8-27B (mean)	66.94	68.36	67.64	72.92	46.44
+ Online RL	**69.25**	71.42	68.12	75.25	48.69
Δ (points)	+2.31	+3.06	+0.48	+2.33	+2.25

Table 2: **Small-scale online RL in the MobilePA-Bench sandbox (%).** Skills/Overall retain legacy gating. Baseline: three-run mean; Δ: improvement over that mean in percentage points.

5. Conclusion

We introduced **MobilePA-Bench** to evaluate central mobile planners across Tool Use, Memory Usage, Skill Usage, and Sub-agent Collaboration in a shared stateful environment. Evidence-aligned checkers and setting-specific capability gates connect task outcomes with resource and procedural requirements, supporting diagnosis beyond aggregate success. The experiments show that retrieving memories, loading skills, or selecting a sub-agent does not by itself satisfy the full task requirements. An online-RL baseline also improves all four capability scores under the reported protocol, providing initial evidence that the sandbox can support planner optimization as well as evaluation. Together, these results establish a testbed for identifying mobile planning weaknesses and assessing targeted improvements.

Limitations

MobilePA-Bench evaluates central planning through supplied tool interfaces in a simulated environment. It does not measure visual grounding, low-level GUI execution, or real-device variability.

Sub-agent evaluation covers planner-side routing and handoff rather than downstream specialist performance; transfer to real-device workflows remains unverified. The four capabilities are evaluated on distinct task subsets, rather than jointly in every trajectory. Coverage is finite, especially for the 89 Sub-agent tasks; difficulty is not calibrated across subsets. Gates verify observable compliance, not causal use of memory or skill contents. Overall is a benchmark index, not a user-experience ranking.

References

[1] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. AndroidWorld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2024. URL https://arxiv.org/abs/2405.14573.

[2] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024. URL https://arxiv.org/abs/2404.07972.

[3] Weikai Xie, Li Zhang, Shihe Wang, Rongjie Yi, and Mengwei Xu. DroidCall: A dataset for LLM-powered Android intent invocation. *arXiv preprint arXiv:2412.00402*, 2024. URL https://arxiv.org/abs/2412.00402.

[4] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *ACL*, 2024. URL https://arxiv.org/abs/2407.18901.

[5] Victor Barres, Ben Shi, Ola Zytek, Soham Ray, Keshav Dhandhania, and Pedram Razavi. τ^3-bench: Advancing agent benchmarking to knowledge and voice. Sierra Research, March 2026. URL https://sierra.ai/blog/bench-advancing-agent-benchmarking-to-knowledge-and-voice.

[6] Junlong Li, Wenshuo Zhao, Jian Zhao, Weihao Zeng, Haoze Wu, Xiaochen Wang, Rui Ge, Yuxuan Cao, Yuzhen Huang, Wei Liu, Junteng Liu, Zhaochen Su, Yiyang Guo, Fan Zhou, Lueyang Zhang, Juan Michelini, Xingyao Wang, Xiang Yue, Shuyan Zhou, Graham Neubig, and Junxian He. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. In *ICLR*, 2026. URL https://openreview.net/forum?id=z53s5p0qhf.

[7] Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian J. McAuley, Yejin Choi, and Alex Pentland. Memoryarena: Benchmarking agent memory in interdependent multi-session agentic tasks. *arXiv preprint arXiv:2602.16313*, 2026. URL https://arxiv.org/abs/2602.16313.

[8] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran

Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han-chung Lee, and Dawn Song. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026. URL https://arxiv.org/abs/2602.12670v4.

[9] Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhailong Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. MultiAgentBench: Evaluating the collaboration and competition of LLM agents. *arXiv preprint arXiv:2503.01935*, 2025. URL https://arxiv.org/abs/2503.01935.

[10] Li Zhang, Shihe Wang, Xianqing Jia, Zhihan Zheng, Yunhe Yan, Longxi Gao, Yuanchun Li, and Mengwei Xu. LlamaTouch: A faithful and scalable testbed for mobile ui task automation. *arXiv preprint arXiv:2404.16054*, 2024. URL https://arxiv.org/abs/2404.16054v2.

[11] Wei He, Yueqing Sun, Hongyan Hao, Xueyuan Hao, Zhikang Xia, Qi Gu, Chengcheng Han, Dengchang Zhao, Hui Su, Kefeng Zhang, Man Gao, Xi Su, Xiaodong Cai, Xunliang Cai, Yu Yang, and Yunke Zhao. VitaBench: Benchmarking llm agents with versatile interactive tasks in real-world applications. *arXiv preprint arXiv:2509.26490*, 2025. URL https://arxiv.org/abs/2509.26490v2.

[12] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *ICLR*, 2025. URL https://arxiv.org/abs/2410.10813.

[13] Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *ICML*, 2025. URL https://openreview.net/forum?id=2GmDdhBdDk.

[14] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *ICLR*, 2025. URL https://openreview.net/forum?id=roNSXZpUDN.

[15] Zixiang Liu, Wenrui Liu, Elsie Dai, Wenhan Yu, Lei Yu, Tong Yang, Jinjun Han, and Hong Gao. MCPAgentBench: A real-world task benchmark for evaluating llm agent mcp tool use. *arXiv preprint arXiv:2512.24565*, 2025. URL https://arxiv.org/abs/2512.24565.

Appendix

A. Benchmark and Tool Statistics

We first summarize the query set and tool inventory. Query domains describe user tasks, whereas tool domains organize the environment's executable APIs; the two taxonomies serve different purposes.

A.1. Dataset Composition and Query Coverage

MobilePA-Bench contains 1,705 distinct queries: 1,040 for Tool Use, 376 for Memory Usage, 200 for Skill Usage, and 89 for Sub-agent Collaboration. The same 200 Skill queries are evaluated under both SOR and MTSR, resulting in 1,905 evaluation instances per model rather than additional distinct queries.

Figure 6 summarizes all 1,705 tasks under an AI-assisted descriptive taxonomy of eight query domains and 30 subclasses. This coverage taxonomy is descriptive and does not define the four official capability scores.

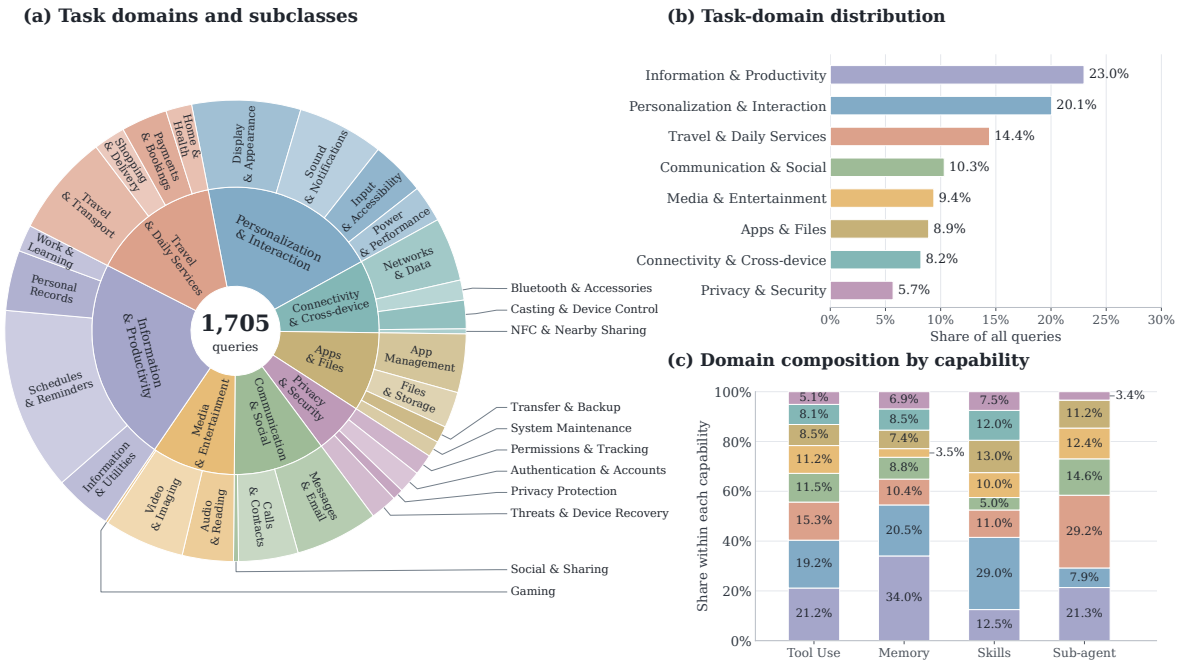


Figure 6: **Descriptive coverage of 1,705 queries.** (a) Query domains and subclasses; (b) domain shares; (c) domain composition within each capability, normalized to 100%. Colors are consistent across panels; stacks in (c) follow (b) from bottom to top.

A.2. Memory Task Statistics

Figure 7 characterizes the 376 Memory Usage tasks by memory reasoning type, target operation, and application domain. These complementary views describe the information a task requires, the operation it targets, and its application context.

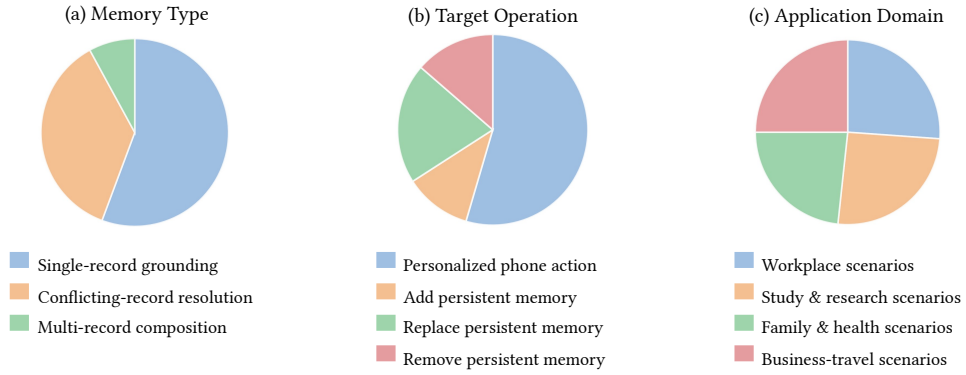


Figure 7: **Coverage of the 376 Memory Usage tasks along three diagnostic axes:** (a) memory reasoning type (single-record grounding, conflicting-record resolution, and multi-record composition); (b) target operation (personalized phone actions and memory addition, replacement, or removal); and (c) application domain.

Domain	# Tools	Description
Audio & Entertainment	25	Media control, playback, camera, screen recording, and music recognition.
Apps & Storage	23	App lifecycle, installations, permissions, notifications, and storage cleanup.
Display & Sound	22	Brightness, volume, sound modes, dark mode, DND, and power schedules.
System Settings	22	Control center, device info, desktop layout, language, and system updates.
Time Management	16	Calendar schedules, tasks, alarms, timers, and automation rules.
AI Assistant	16	GUI sub-agent routing, image processing, memory search, and smart perception.
Calls & Communication	15	Contacts, calls, SMS/email handling, and harassment interception.
Network & Connectivity	14	WLAN, Bluetooth, mobile data, hotspot, NFC, and pairing management.
Travel & Lifestyle	13	Weather, navigation, transit tracking, payment codes, and ticket booking.
Devices & Cross-device	13	Screen casting, multi-screen sharing, device migration, and pairing logs.
Input & Interaction	12	Screenshots, QR scanning, air gestures, system buttons, and AOD modes.
Utilities & Productivity	11	Browser, calculator, conversions, translations, and open-domain QA.
Security & Privacy	10	Screen lock, app locks, biometrics, password vault, and emergency alerts.

Table 3: Tool domains in the MobilePA-Bench mobile function-call environment.

A.3. Tool Inventory by Domain

Table 3 lists the 212 tools across 13 tool domains. This inventory describes the available action interfaces, not the query-domain distribution in Figure 6.

B. Extended Experimental Results

B.1. Evaluation Configuration and Provenance

The reported results use matched task definitions and a common aggregation policy for all 14 models. Each model has 1,040 Tool Use trajectories, 376 Memory Usage trajectories, 89 Sub-agent Collaboration trajectories, and 400 Skill Usage trajectories obtained by executing the same 200 tasks under SOR and MTSR. Thus, the 1,705 distinct tasks produce 1,905 scored trajectories per model. API-judged outcomes use the same two-judge unanimous-AND policy throughout; saved database results and

required memory or skill gates are retained where applicable.

The results follow the setting-specific Skill criteria in Appendix E: SOR requires an acceptable skill route and task completion, whereas MTSR accepts valid direct-tool, skill-based, or mixed completions without a loading gate. Reported Skills and Overall scores, ranking comparisons, and online-RL results use these criteria. Route-conditioned diagnostics are separate from MTSR task success.

Before aggregation, we checked row counts, task identifiers, query text, and evaluation-policy fields across the canonical result manifests. These checks establish matched benchmark content and scoring policy, but do not by themselves prove byte-identical historical tool implementations, initial-state serialization, or checker code. Missing or invalid predictions remain in the full denominator as failures.

Candidate actions. Candidate schemas are supplied with each task, not produced by a learned online retriever in this evaluation. Non-Skill inputs expose 15 tools per task. SOR exposes 20 skills without direct business tools; MTSR exposes 15 direct tools and five skills. All 14 models' recorded Skill candidate lists confirm these sizes and contain at least one complete annotated route for each task. Loading a skill adds its bound tools to the active action set. These configurations differ in both tool access and skill candidates, so their comparison is not a single-factor direct-tool ablation.

An offline audit of the task inputs found that every tool named in nonempty reference calls is present among the supplied candidates: 841/841 Tool Use tasks, 89/89 Sub-agent tasks, and 176/176 single-turn plus 200/200 multi-turn Memory tasks. The remaining 199 Tool Use tasks have empty reference-call lists and are not counted as successful recall cases. Skill inputs do not provide an independent reference-call list, so we report route availability rather than business-tool recall for them. These checks establish access to the recorded reference tools or routes, not the necessity of those references, coverage of all valid solutions, or general retrieval quality. Historical Skill lists are checked against saved trajectories; input-level coverage does not establish byte-identical runtime schemas for every historical run.

Model identities and reproducibility scope. The additional `Qwen3.8-27B` model used for stability and exploratory training is not the `Qwen-3.8-Max` entry in the main comparison. Historical task-ID and policy matching do not establish access to identical future API snapshots. The main results retain the recorded judgments and execution outcomes; the main-text weighting comparisons hold these records fixed.

B.2. Full-Cohort Capability Summaries

Figure 8 complements the seven-model profiles in Figure 5 with aggregate diagnostics for all 14 models. Panels (a), (b), and (d) summarize means and model ranges; panel (c) reports paired MTSR–SOR differences in the Skill Joint diagnostic as point estimates. These summaries retain the full cohort rather than selecting one representative per model family.

Across the full cohort, mean Tool Use accuracy is 87.3% for Reference Tracking and 81.0% for Intent Management, compared with 72.1% for Parameter Grounding, 70.3% for Dependency Planning, and 70.1% for Boundary Handling. Memory's mean Retrieval, Gate, conditional completion, and Joint rates are 83.8%, 74.0%, 89.2%, and 66.1%; their different denominators preclude interpreting them as one attrition funnel. Every model's Skill Joint diagnostic is lower in MTSR than in SOR. The largest decreases occur for `Qwen-3.7-Max` (55.5 percentage points), `GLM-5.2` (42.0), and `Gemini-3.1-Pro` (39.5). These route-conditioned differences do not measure differences in setting-specific task success. Mean Sub-agent Route, route execution, Task Proxy, and Joint rates are 79.5%, 77.2%, 65.4%, and

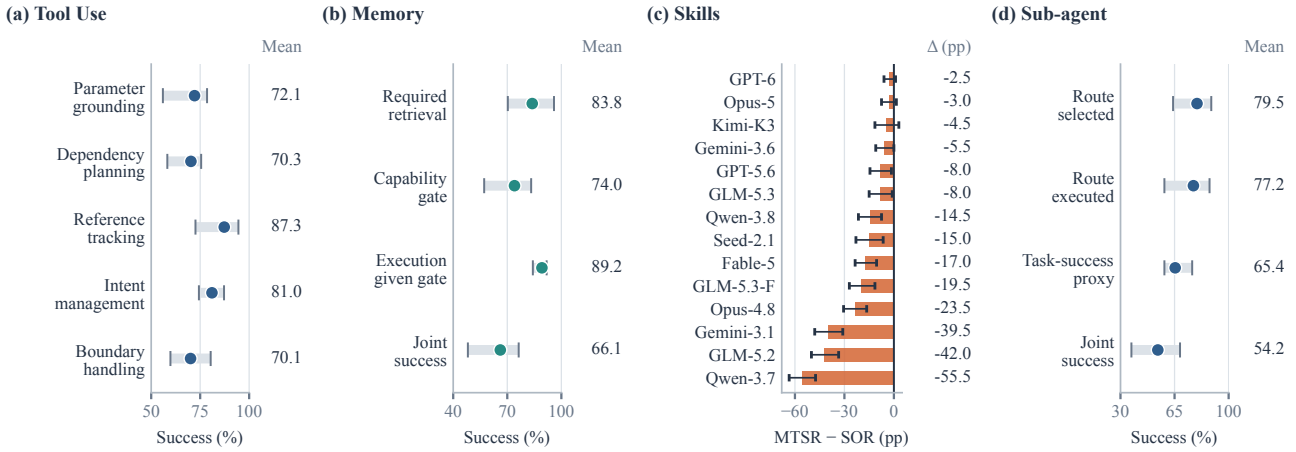


Figure 8: **Capability diagnosis across 14 models.** (a,b,d): unweighted means and model ranges (not confidence intervals); axes start above zero. (c): paired Skill Joint-diagnostic differences (MTSR–SOR), shown as point estimates. The Sub-agent task-success proxy is not an independent handoff-quality measure.

54.2%, respectively. These summaries describe the fixed 14-model cohort, not confidence intervals over models.

B.3. Exploratory Online RL

An online-RL checkpoint initialized from `Qwen3.8-27B` was trained through the simulated mobile environment using 4,000 training queries disjoint from the benchmark's test queries. Table 2 compares its scores with the original model's mean over three complete evaluations. The reported Overall improvement is 2.31 percentage points. Query-level disjointness does not establish separation of templates, profile worlds, or tool schemas. The comparison uses one trained checkpoint rather than repeated independent training runs and therefore remains exploratory.

C. Mobile Environment Details

This appendix provides the action-interface definitions and sandbox example supporting Section 3.1. The tool inventory is reported in Appendix A.3.

C.1. Unified Action Interfaces

All planner resources are represented as structured function schemas with a unique name, textual description, and JSON argument schema:

- **Direct Mobile Tools** query or mutate mobile-system state across domains such as messaging, settings, media, and calendars.
- **Memory Tools** search persistent user-profile and contextual records and return ranked results as structured feedback.
- **Skill Loading Tools** return procedural instructions and expose the concrete tools associated with a reusable skill.
- **Sub-agent Entry Tools** route a task and its contextual payload to a specialized downstream agent.

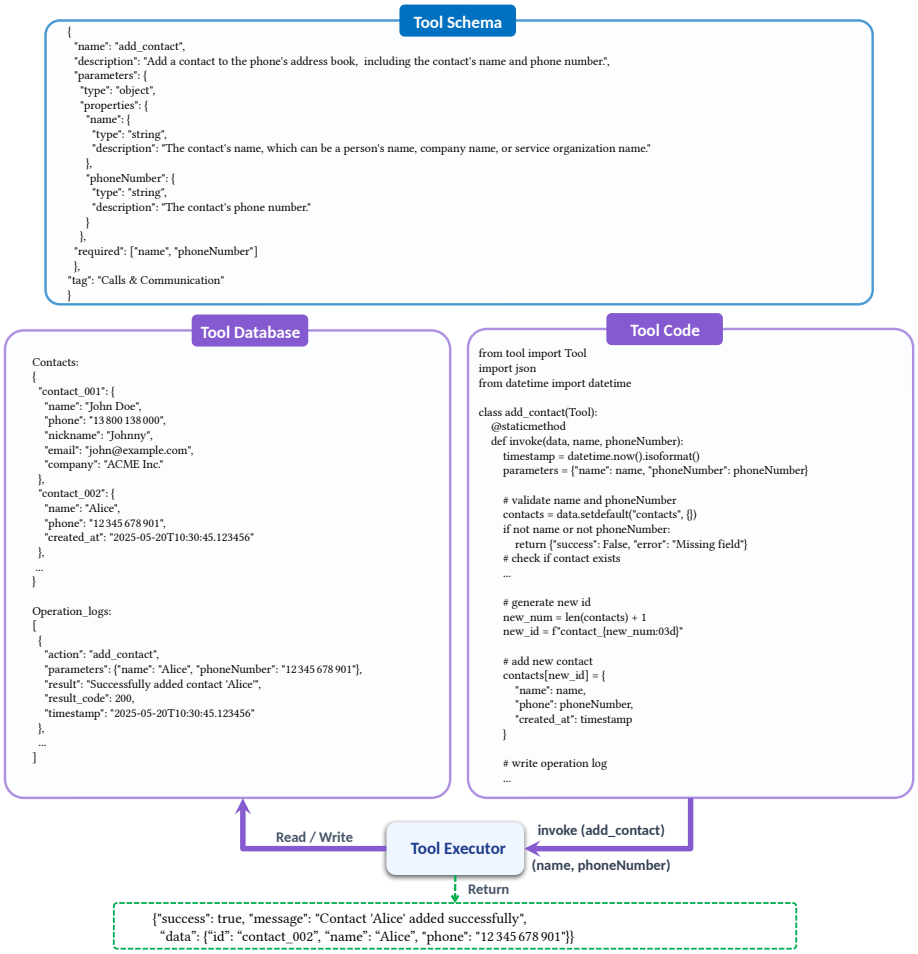


Figure 9: **Stateful sandbox architecture for an `add_contact` invocation.** A schema defines the interface, implementation code validates and executes the call, and a shared database records state changes and operational logs. The executor returns structured feedback to the planner.

Let $\mathcal{G}$ denote the catalog of direct, memory, and sub-agent tools, and let $\mathcal{L}_q$ denote the candidate skill loaders for query q. The supplied task configuration defines a candidate tool set $C_q \subseteq \mathcal{G}$; the initial action set is

$$\mathcal{A}_0 = C_q \cup \mathcal{L}_q. \tag{4}$$

This notation does not assume a learned online retriever. In SOR, C_q is empty and $|\mathcal{L}_q| = 20$; in MTSR, $|C_q| = 15$ and $|\mathcal{L}_q| = 5$. Loading skill s expands the next-step action set with the tools bound to that skill:

$$\mathcal{A}_{t+1} = \mathcal{A}_t \cup \mathcal{G}(s), \tag{5}$$

while the action set remains unchanged when no skill is loaded.

C.2. Sandbox Execution Example

Figure 9 illustrates how the shared schema, implementation, and state layers instantiate an `add_contact` call.

D. Capability-Specific Task Construction

Section 3.2 describes the motivation and construction principle for each capability. Here we provide the detailed behavioral categories, action-space settings, and annotation scope used to instantiate those tasks.

Tool Use. Tool Use tasks are synthesized from human-curated mobile seeds and introduce realistic execution friction through reference obfuscation, permission blocks, missing arguments, and intermediate state changes. The five behavioral categories are:

- **Tool and Parameter Grounding**: selecting the target API and resolving explicit or implicit arguments;
- **Conditional and Dependency Planning**: satisfying prerequisite order and preserving cross-step dependencies;
- **Reference and State Tracking**: resolving historical entities, references, and evolving application state;
- **Intent Revision and Task Management**: handling corrections, target changes, multi-intent requests, and continuation; and
- **Boundary Detection and Error Recovery**: recognizing capability limits and recovering from contradictory constraints or runtime errors.

These categories are mutually exclusive and contain 377, 249, 73, 117, and 224 tasks, respectively. Deterministic operations use Tool Call verification, whereas path-equivalent workflows use State Change verification.

Memory Usage. Memory tasks are derived from coherent profile worlds containing user habits, identities, routines, and historical preferences. Requests intentionally omit relevant personal information, and each example records the memory IDs required to resolve the request together with its downstream execution objective. Appendix A.2 characterizes these tasks by memory reasoning type, target operation, and application domain.

Skill Usage. Composite mobile routines are organized into reusable skill packages. Each task specifies an execution objective and one or more acceptable skill routes. In *Skill-Only Routing (SOR)*, skill loaders are initially available without direct business tools; all skills in at least one acceptable route must be loaded before business-tool use, and the task must then be completed. *Mixed Tool-Skill Routing (MTSR)* exposes candidate skill loaders together with supplied direct tools. It accepts valid direct-tool, skill-based, or mixed solutions within the available action space, without requiring an annotated route. Reference routes support SOR verification and route diagnostics in both settings, rather than defining a unique solution for MTSR.

Sub-agent Collaboration. Sub-agent tasks cover specialized routes such as GUI manipulation, visual processing, conditional monitoring, and interactive practice. Annotations specify the valid downstream route and the contextual payload required for handoff. Evaluation stops at routing and handoff quality and does not score the downstream sub-agent policy.

E. Verifier and Capability-Gate Details

E.1. Primary Checkers

Each query receives exactly one primary checker before model evaluation.

Tool Call. This checker is used when the annotated solution requires a deterministic operational sequence. It verifies required tool names, call order, argument fields, and normalized argument values, while rejecting extraneous calls with task-relevant side effects.

State Change. This checker is used when multiple action sequences may reach the same valid outcome. It compares the terminal database transition $\mathcal{D}_T - \mathcal{D}_0$ with the annotated target delta and rejects destructive or unrelated writes.

Agent Behavior. This checker is used when neither an exact call sequence nor a static database mutation fully captures completion. Sub-agent tasks use two explicit rubric fields: *Call Sub-agent* checks route selection, and *Appropriate Follow-up Behavior* checks the handoff payload and subsequent response. Other task-specific rubrics inspect necessary clarification or presentation of a user-facing result.

E.2. Memory and Skill Gates

Memory retrieval and action compliance. Let $\mathcal{M}_q^*$ denote the annotated memory IDs required by query q. The set returned by memory search over the trajectory is

$$\widehat{\mathcal{M}}_T = \bigcup_{\substack{t:\, a_t = \\ \texttt{search_user_memory}}} \text{MemoryIDs}(f_t). \tag{6}$$

On retrieval-required tasks, the retrieval check passes only if all required IDs occur in the recorded search results and those results are valid. Partial retrieval is not a task-level pass. The reported Retrieval rate uses the 367 tasks requiring this check, not all 376 Memory tasks.

The complete Memory gate additionally includes the task's declared action constraints. Let Q_q contain the required checker predicates, including the retrieval check when applicable, and let a_q denote the action-policy verdict. Then

$$g_{\text{mem}}(q) = a_q \wedge \bigwedge_{h \in Q_q} h(q). \tag{7}$$

An empty conjunction and an absent action policy each evaluate to one; required checks with missing or invalid evidence fail. Where specified, the action policy checks required actions and arguments, forbids unauthorized state-changing calls, and applies the task-declared allowances for read-only helpers or recoverable attempts. These checks supplement, rather than replace, database-state or behavior-based completion checks.

Memory addition, replacement, and removal tasks without a retrieval requirement are evaluated by their fixed primary checker and any applicable action constraints. Gate and Joint rates use all 376 tasks. The retrieval check scans the whole trajectory and does not impose retrieval-before-action ordering. It establishes observable access to the required records, not causal use of their contents.

Memory metric definitions. For each model, let $N_R = 367$ be the number of retrieval-required tasks and n_R the number passing the retrieval check. Across all $N = 376$ Memory tasks, let n_G count complete-gate passes and $n_{G\cap T}$ count tasks passing both the gate and the primary completion checker. In fractional units, the four reported rates satisfy

$$\begin{aligned} \text{Retrieval} = \frac{n_R}{N_R}, \quad \text{Gate} = \frac{n_G}{N}, \\ \text{Exec.} \mid \text{Gate} = \frac{n_{G\cap T}}{n_G} \quad (n_G > 0), \\ \text{Joint} = \frac{n_{G\cap T}}{N} \\ = \text{Gate} \times (\text{Exec.} \mid \text{Gate}). \end{aligned} \tag{8}$$

Joint is the Memory score reported in Table 1. Retrieval uses a narrower check and a different task subset, so it cannot replace Gate in this identity. The identity holds per model before rounding; it does not require the product of the cross-model means in Figure 8(b) to equal mean Joint.

Ordered skill routing in SOR. Let $\mathcal{P}_q$ be the set of acceptable skill routes for query q, where each route $P \in \mathcal{P}_q$ is a set of skills that must all be loaded to satisfy that route. For setting $z \in \{\text{SOR}, \text{MTSR}\}$, let $\ell_{q,z}(s)$ be the first recorded load position of skill s, and let $b_{q,z}$ be the position of the first non-skill tool call. Missing loads and the absence of a non-skill call are assigned position $+\infty$. The ordered-route check is

$$g_{\text{skill}}(q, z) = \mathbf{1}\left[\exists P \in \mathcal{P}_q : \max_{s \in P} \ell_{q,z}(s) < b_{q,z}\right]. \tag{9}$$

In SOR, this check is mandatory: at least one complete route must be loaded before any non-skill tool call; the skills within a route need not be loaded in a prescribed order, and every alternative route need not be loaded. Non-skill calls include read-only and failed calls, not only successful state changes. A load is recorded when the harness resolves a skill-loading call to an available skill. Merely mentioning a skill or loading it after a business call does not satisfy the gate. When available, call sequence indices determine order; historical records containing only turn indices use the evaluator's within-turn skill-first convention. Missing ordering evidence does not pass.

Setting-specific Skill success. Let $v_{q,z}$ denote execution validity and $j_{1,q,z}, j_{2,q,z}$ the two judges' task-completion verdicts for the trajectory in setting z. Define completion as

$$T_z(q) = v_{q,z} \wedge j_{1,q,z} \wedge j_{2,q,z}. \tag{10}$$

The success criterion is

$$\text{Succ}_z(q) = \begin{cases} g_{\text{skill}}(q, z) \wedge T_z(q), & z = \text{SOR}, \\ T_z(q), & z = \text{MTSR}. \end{cases} \tag{11}$$

Execution-validity checks reject model errors, unparseable business-tool arguments, calls outside the effective tool set, and empty trajectories. An ordinary tool failure is not by itself an automatic rejection: the judges assess whether subsequent recovery, clarification, or refusal appropriately addresses the task. MTSR rubrics assess the requested outcome and task-declared constraints, not whether the planner loads an annotated skill, follows its route order, or reproduces a reference trajectory. Calling available tools directly, using loaded skills, or combining both are all valid paths if the task is completed. Loading a skill alone is never sufficient, and omission of skill loading alone is not an MTSR failure.

For diagnostics, Route reports g_{skill}, Exec.|Route reports completion conditional on passing that check, and Joint reports their conjunction. Joint equals SOR success, but is only an ordered-route diagnostic

in MTSR, not its success rate. A failed ordered-route check does not by itself establish that no skill was used.

Skill score aggregation. Let Q be the shared set of $N = 200$ Skill queries. We compute each setting's success rate on the full query set and retain their equal average as the Skills score:

$$\begin{aligned} S_z &= \frac{1}{N} \sum_{q \in Q} \mathrm{Succ}_z(q), \\ S_{\text{Skills}} &= \frac{S_{\text{SOR}} + S_{\text{MTSR}}}{2}. \end{aligned} \tag{12}$$

Scores are displayed as percentages. Missing or invalid predictions remain in the denominator as failures. This aggregate measures completion under two resource configurations; an MTSR success is not necessarily evidence of skill use. The reported Skills scores use these setting-specific success rates, as described in Appendix B.1.